

Bridger Burkes

Final Submission

MART 391

5/9/2024

Timeline:

February - Switching Unity Netcode to Mirror

March - Basic Gameplay, map design

April - Models/graphics

May - Fine tuning, security

Overview:

Formula2

A competitive multiplayer racer similar to Wipeout 2048. Formula2 aims to be a top down racer with some combat elements, but highly reliant on a player to control the extremely fast craft.

Instruction:

To login/start a server, insert the following usernames and passwords to create 2 clients and a server (can work with more players but only 2 spawn points are set up as well as two logins, also does not work with host setting, only clients and servers can work.)

Username (same as passwords):

player1

player2

player3

Create a server and two clients. Press p on both clients to start a race. WASD for movement and shift for airbrakes (fast turning and deceleration)

Challenges:

The biggest issues I had were setting up collisions to work across all clients, as well as syncing clients correctly. In the end, collisions required using generative AI to figure out where the issue was stemming from, and removing several unneeded pieces of code that compromised collision physics. Syncing was largely solved by switching from netcode to Mirror, as Mirror offered much more flexibility than netcode. I still suffered the same issue twice, however, where the host doesn't sync across all clients correctly, or cause issues when attempting to sync clients. I suspect this has to do with the fact of its dual roles as both a client and a server, thus incorporating both into a single player role, where my code separated the two into distinct entities. I still have no answer on how to fix this issue, but it doesn't seem to affect gameplay in any way as long as the host strictly plays as a server, and not a client.

Time Reflection:

Most of the timeline is correct, up until April. I ran into some issues with collision which prevented me from devoting my attention to models, as well as map design, putting those aspects into the May category. Thus fine tuning and security were not implemented beyond their original implementations. Map design was also hard to find a time to implement, as creating a detailed map also meant creating a custom collision box for each part. While I couldn't implement a map,

I have a plan on how to later implement it if I could. This would involve making track pieces for each part, allowing for considerably less intricate collision systems than a custom blender model and further custom Unity collider.

Generative AI:

AI was only used on solving collision issues, which meant some additions and manipulations of player movement. This did lead to a considerably more complex movement system than I could have created. That said, with every prompt I ask for a detailed explanation on how the code works, if I am not already able to recognize its usage. This provides me the capability to later manipulate the code without resorting to using generative AI again. To get back on track, this did solve my collision system, however it required a majority of effort on my part as many of the reasons for improper collision had to do with my player parameters within the editor, and not the script.

As for why I used generative AI, is mainly because the problem had spiraled to a point to where I couldn't solve the issue. I had tried every solution I could think of, whether by changing how movement is applied, changing the collision system, to implementing a raycast system where collision was predictive. None of this worked and I decided to use AI to find the issue. This required some script changes, as well as in-editor changes to how the player handled collisions.